RCA Module Explanation

contribution.py, dimension_analysis.py, and tree.py

This document explains the three core root-cause-analysis modules in backend/app/analysis/rca. Together they answer three questions: what SQL should be run, how each segment's movement should be attributed, and which branches deserve a deeper drill-down.

The engine module orchestrates execution, but these files deliberately keep their own responsibilities narrow. dimension_analysis.py builds DuckDB SQL strings, contribution.py turns query results into evidence-rich driver nodes, and tree.py decides how to descend through dimensions.

1. Big Picture

The RCA flow starts by projecting the source relation into a narrow temporary table named rca_base. That temp table keeps only timestamp, measure, and selected dimensions. The engine can then aggregate periods and dimensions without repeatedly scanning the full source.

After SQL returns totals per segment, contribution.py decides whether the KPI can be decomposed at all. SUM and COUNT use net-change attribution, AVG uses a mix/rate decomposition, COUNT_DISTINCT is only allowed when overlap checks prove the dimension partitions the distinct key set, and MIN, MAX, and MEDIAN are treated as unattributable for additive driver ranking.

tree.py then converts a flat driver list into a drill-down plan. It picks the most explanatory remaining dimension, expands only material primary nodes, and records stop reasons when a node should not be queried further.

Key Points

- dimension_analysis.py: safe SQL builders and period/dimension aggregation shape.
- contribution.py: attribution basis, contribution math, support flags, and driver node construction.
- tree.py: dimension selection, branching limits, drill eligibility, unexplained share, and synthetic root.

2. dimension_analysis.py

This file is a SQL factory. Every public function returns SQL text, and build_filter_clause returns a predicate plus bind parameters. It does not open a connection or execute anything; the engine owns execution.

build_base_table_sql creates a temp table with columns ts, m, and d0..dN. That projection is the performance foundation: downstream RCA queries work on a narrow, in-memory DuckDB table instead of the original wide dataset.

period_case_expression assigns each row to current, previous, or neither using half-open time windows. Half-open comparisons avoid double-counting records that land exactly on a boundary.

build_totals_sql aggregates the current and previous periods in one pass using FILTER clauses. It returns values, non-null measure counts, row counts, and rows outside the compared periods.

build_breakdown_sql performs a UNION ALL unpivot over all requested dimensions, groups by dimension and segment, and returns both periods side by side. This gives full-outer semantics naturally: new segments and lost segments remain visible because grouping happens across both periods.

build_distinct_overlap_sql supports COUNT_DISTINCT attribution. It computes sum(per-group distinct) - overall distinct for each period. A zero gap means the chosen dimension partitions the distinct key set and can be decomposed safely.

build_filter_clause is the safety gate for KPI filters. It accepts a closed operator grammar, verifies that columns exist, quotes identifiers, and emits placeholders with bind params instead of allowing raw SQL passthrough.

Reference Flow

```
Source -> build_base_table_sql -> rca_base(ts, m, d0..dN)
rca_base -> build_totals_sql -> global period totals
rca_base -> build_breakdown_sql -> per-dimension SegmentTotals
rca_base -> build_distinct_overlap_sql -> COUNT_DISTINCT additivity check
```

3. contribution.py

This file is the mathematics layer. It contains pure functions over SegmentTotals and Totals. There is no SQL and no I/O, which makes the analytical behavior easier to test and reason about.

percent_change computes signed percent change using abs(previous) as the denominator. That matters for negative-valued KPIs: moving from -100 to -150 should read as -50%, not +50%.

basis_for decides the attribution basis. Non-decomposable aggregations get an unattributable reason. If total net change is tiny compared with gross segment movement, it switches to gross movement to avoid thousands-of-percent artifacts caused by cancellation. AVG uses MIX_RATE, while additive aggregations use NET_CHANGE.

_avg_effects implements a centered Bennet decomposition for averages. It splits the total average movement into rate effect, meaning a segment’s own average changed, and mix effect, meaning the segment’s share of volume changed. Centering by the overall average prevents fake effects when all groups have the same mean.

build_nodes turns SegmentTotals into DriverNode objects. Each node carries raw values, absolute and percent change, contribution, rate/mix effects when applicable, current and previous shares, expected proportional change, excess change, new/lost flags, and support warnings.

A key design rule is that contribution is always relative to the global KPI change, even for deeper tree nodes. The local relationship is stored separately as share_of_parent_change. This keeps a child from appearing as 100% of the RCA just because it explains all of a small parent.

contribution_sum is an invariant check. Under NET_CHANGE and MIX_RATE, contributions should sum to 1. Under GROSS_MOVEMENT, absolute contribution magnitudes should sum to 1. Drift signals missing rows or an unreported residual bucket.

explanatory_power ranks dimensions by deviation from proportional movement rather than by the largest single segment. This avoids always rewarding high-cardinality dimensions that happen to contain one big value.

Reference Flow

```
NET_CHANGE: contribution = segment_delta / global_delta
GROSS_MOVEMENT: contribution = segment_delta / sum(abs(segment_delta))
MIX_RATE: contribution = (rate_effect + mix_effect) / global_delta
Expected change: level_change * previous_segment_share
Excess change: actual_segment_change - expected_change
```

4. tree.py

This file owns drill-down decisions. It does not execute SQL; it tells the engine what to ask next and why the tree should stop expanding in particular places.

select_dimension compares candidate breakdowns for a node. It ranks normal partition splits by explanatory power, then uses strongest contribution, number of nodes, and original dimension order as tie-breakers. A dimension with only one populated segment is handled as a pure split, because it can be informative even though its explanatory-power score is zero by construction.

branching_at returns how many nodes can expand at each depth. The configured shape is 3 at depth 1, then 2 at deeper levels. This prevents the RCA tree from exploding into a wide, low-signal report.

drillable records a stop_reason directly on nodes that should not be expanded. Reasons include max_depth_reached, no_dimensions_left, residual_bucket, contribution_immaterial, and insufficient_rows for mean-like aggregations.

frontier_for picks the primary nodes worth expanding at the current depth, sorted by contribution magnitude. Nodes beyond the branch limit are marked with branching_limit.

unexplained_share compares a parent node’s absolute change with the sum of its children. This is especially useful when a dimension query was truncated or children only partially cover the parent.

synthetic_root creates a depth-0 DriverNode for the whole KPI. It makes tree invariants simple: the root represents the full change, and its children represent decomposed pieces of that same change.

Reference Flow

```
Engine gathers candidate dimension breakdowns
tree.select_dimension chooses the best remaining split
tree.frontier_for chooses material primary nodes
tree.drillable decides whether each node earns another query
tree.unexplained_share records coverage gaps after children attach
```

5. How They Work Together

The cleanest way to read these files is as a pipeline. dimension_analysis.py asks DuckDB for period totals and segment totals. contribution.py converts those totals into normalized, evidence-rich DriverNode objects. tree.py chooses the next split and controls the breadth-first expansion.

The modules also protect important product promises. They avoid ranking by raw percent change, preserve offsetting factors, distinguish new and lost segments, avoid fake average effects, surface unattributable aggregations honestly, and carry residual or unexplained shares instead of hiding uncertainty.

The result is an RCA engine whose output can say not only what moved, but also how that claim was calculated, where the decomposition is mathematically valid, and where the investigation deliberately stopped.

Key Points

- Performance promise: one narrow temp table and compact aggregate queries.
- Math promise: contributions are shares of the net KPI change unless cancellation requires gross movement.
- Evidence promise: each node keeps counts, shares, support flags, expected change, and excess change.
- Tree promise: deeper analysis is bounded, material, and explicit about stop reasons.